Sensensble Geometry: A Web-Native Lab Console for Geometric Algebra and Geometric Neural Computing

Intelligent Interfaces Group

***Abstract*—We present Sensensble Geometry, a web-native lab console for interactive geometric algebra computation and geometric neural computing (Geometric NC). The platform implements a full $\mathrm{Cl}(3,0)$ Clifford algebra engine in Rust, compiled to WebAssembly for client-side multivector arithmetic at native speed. A reactive Svelte frontend couples a CodeMirror editor to a Three.js rendering canvas, enabling real-time "Code-to-Canvas" synthesis where user-authored R or Python scripts are parsed and projected into a 3D geometric scene. For inference-heavy workloads, a monolithic Go gateway multiplexes WebSocket connections from the client and dispatches multivector state over gRPC to Python worker nodes running PyTorch and JAX. We describe the mathematical foundations of the platform, present the implementation of each architectural layer, verify the correctness of the Clifford engine against known algebraic identities, and outline the path toward integrating Equivariant Neural Networks ($E(n)$-GNNs) and Clifford Neural Layers into the worker backend.**

***Index Terms*—Geometric Algebra, Clifford Algebra, Multivector, WebAssembly, Equivariant Neural Networks, gRPC, Interactive Simulation**

I. Introduction

The representation of 3D space in machine learning and physical simulation remains dominated by matrix algebra and Euler angle parameterizations. Despite their ubiquity, these tools introduce well-documented pathological discontinuities. Euler angles suffer from gimbal lock—the loss of a rotational degree of freedom when two rotation axes align [5]. Quaternions avoid gimbal lock but lack a natural way to represent planes, volumes, or higher-dimensional geometric objects, forcing practitioners to maintain separate formalisms for rotations, reflections, and projections.

Geometric Algebra (GA), also known as Clifford Algebra, provides a single, coordinate-free algebraic framework that unifies scalars, vectors, oriented planes (bivectors), and oriented volumes (trivectors) into a single object: the *multivector* [1]. Recent work in machine learning has demonstrated that neural networks operating directly on multivectors—so-called Geometric Neural Computing (Geometric NC)—achieve natural $E(n)$-equivariance, outperforming standard architectures on molecular dynamics, point cloud classification, and n-body simulation tasks [3], [4].

In this paper, we present Sensensble Geometry, a platform designed to make these mathematical structures computationally tangible and interactively explorable. The platform operates across four layers:

1) A **Rust/WebAssembly Clifford engine** implementing $\mathrm{Cl}(3,0)$ with geometric, wedge, inner, and dual products.
2) A **reactive Svelte frontend** coupling a code editor to a Three.js 3D scene via a regex-based Code-to-Canvas parser.
3) A **monolithic Go gateway** handling authentication, REST, and WebSocket-to-gRPC dispatch.
4) A **Python gRPC worker** backend designed to host PyTorch/JAX Clifford Neural Layers.

II. Fundamentals of Geometric Algebra

We briefly review the algebraic structures underlying the platform. A comprehensive treatment can be found in Dorst, Fontijne, and Mann [2].

A. The Geometric Product

Given a vector space $\mathbb{R}^n$ with orthonormal basis $\{e_1, \ldots, e_n\}$ satisfying $e_i e_j + e_j e_i = 2\delta_{ij}$, the *geometric product* of two vectors $a, b \in \mathbb{R}^n$ decomposes as:

$$ab = a \cdot b + a \wedge b \tag{1}$$

where $a \cdot b = \frac{1}{2}(ab + ba)$ is the symmetric inner product (a scalar) and $a \wedge b = \frac{1}{2}(ab - ba)$ is the antisymmetric outer product (a bivector representing the oriented plane spanned by a and b) [1].

B. Multivectors in $\mathrm{Cl}(3,0)$

In three dimensions, the algebra $\mathrm{Cl}(3,0)$ is generated by $\{e_1, e_2, e_3\}$ with signature $(+,+,+)$. The full algebra has $2^3 = 8$ basis elements, and a general *multivector* is:

$$\begin{aligned} M = \underbrace{\alpha}_{\text{grade 0}} + \underbrace{\beta_1 e_1 + \beta_2 e_2 + \beta_3 e_3}_{\text{grade 1}} \\ + \underbrace{\gamma_{12} e_{12} + \gamma_{23} e_{23} + \gamma_{31} e_{31}}_{\text{grade 2}} + \underbrace{\delta\, e_{123}}_{\text{grade 3}} \end{aligned} \tag{2}$$

where we use the shorthand $e_{ij} \equiv e_i e_j$. This 8-component object is the fundamental data type of the platform, stored as a contiguous `[f64; 8]` array in Rust and a `repeated float` in the gRPC protocol.

C. Rotors and the Sandwich Product

A key advantage of GA over quaternions is that rotations emerge naturally from the algebra without introducing a new number system. A *rotor* R encoding a rotation by angle θ in the plane defined by a unit bivector $\hat{B}$ is:

$$R = \cos\left(\frac{\theta}{2}\right) - \hat{B}\sin\left(\frac{\theta}{2}\right) \tag{3}$$

The rotation of any multivector M is computed via the *sandwich product*:

$$M' = R\,M\,\tilde{R} \tag{4}$$

where $\tilde{R}$ is the reverse of R. This formula is universally valid: it rotates vectors, bivectors, and trivectors alike, without case-splitting. The absence of special-case logic for different geometric objects is exactly what eliminates gimbal lock and what makes GA-based neural layers naturally equivariant [4].

D. Versors

A *versor* is a product of invertible vectors $V = v_1 v_2 \cdots v_k$. Rotors are even versors (products of an even number of vectors). Odd versors encode reflections. The sandwich product $VM\tilde{V}$ generalizes rotations, reflections, translations (in conformal GA), and dilations into a single algebraic operation [2]. The Versor library (`libvsr`) implements this generalized framework for arbitrary metric signatures [6].

E. Worked Example: Rotating a Vector by $90°$

To make the algebra concrete, we rotate the vector $v = 3e_1$ by $\theta = 90°$ in the e_1e_2 plane. As an 8-component array, $v = [0, 3, 0, 0, 0, 0, 0, 0]$.

The rotor is:

$$R = \cos(45°) - e_{12}\sin(45°) = \tfrac{\sqrt{2}}{2} - \tfrac{\sqrt{2}}{2}\,e_{12} \tag{5}$$

which as an array is $R = [\frac{\sqrt{2}}{2}, 0, 0, 0, -\frac{\sqrt{2}}{2}, 0, 0, 0]$, and $\tilde{R} = [\frac{\sqrt{2}}{2}, 0, 0, 0, \frac{\sqrt{2}}{2}, 0, 0, 0]$.

Computing Rv via the `geometric_product` function yields $[0,\ \frac{3\sqrt{2}}{2},\ \frac{3\sqrt{2}}{2},\ 0,\ 0,\ 0,\ 0,\ 0]$. Then $(Rv)\,\tilde{R}$ gives:

$$v' = [0, 0, 3, 0, 0, 0, 0, 0] = 3e_2 \tag{6}$$

confirming that a $90°$ rotation in the e_1e_2 plane maps $e_1 \mapsto e_2$, as expected. This computation is performed entirely within the Rust engine described in Section III.

III. The Clifford Engine (Rust/WebAssembly)

The computational core of the platform is a $\mathrm{Cl}(3,0)$ algebra engine written in Rust and compiled to WebAssembly via `wasm-pack` [7]. This design ensures that multivector arithmetic runs at near-native speed inside the browser, with no server round-trip required for basic algebraic operations.

A. Data Representation

A `Multivector` is a struct containing a single `[f64; 8]` array, with components ordered as:

$$[s,\ e_1,\ e_2,\ e_3,\ e_{12},\ e_{23},\ e_{31},\ e_{123}] \tag{7}$$

This flat layout enables cache-friendly access and straightforward WASM-to-JS interop via `wasm_bindgen` [8].

B. Geometric Product Implementation

The geometric product ab in $\mathrm{Cl}(3,0)$ expands into a 64-term bilinear form determined by the Cayley table of the algebra. The Rust implementation computes all 8 output components in a single pass:

```
// Rust: geometric_product (Cl(3,0))
pub fn geometric_product(&self, other: &Mv)
  -> Mv {
  let (a, b) = (self.data, other.data);
  let mut r = [0.0; 8];
  // Scalar component
  r[0] = a[0]*b[0] + a[1]*b[1] + a[2]*b[2]
       + a[3]*b[3] - a[4]*b[4] - a[5]*b[5]
       - a[6]*b[6] - a[7]*b[7];
  // e1 component
  r[1] = a[0]*b[1] + a[1]*b[0] - a[2]*b[4]
       + a[3]*b[6] + a[4]*b[2] - a[5]*b[7]
       - a[6]*b[3] - a[7]*b[5];
  // ... (6 more components, 64 terms total)
  Multivector { data: r }
}
```

Fig. 1: Abbreviated Rust implementation of the $\mathrm{Cl}(3,0)$ geometric product. Each of the 8 output components is a sum of 8 signed terms derived from the algebra's Cayley table.

The engine also exposes `wedge`, `inner`, and `dual` products, all built on the same contiguous array and exported to JavaScript through `#[wasm_bindgen]`.

C. Correctness Verification

We verify the engine against the defining identities of $\mathrm{Cl}(3,0)$:

1) $e_i^2 = +1$ for $i \in \{1,2,3\}$ (positive-definite signature).
2) $e_ie_j = -e_je_i$ for $i \neq j$ (anticommutativity).
3) $e_1e_2e_3 \cdot e_3e_2e_1 = 1$ (pseudoscalar squared is -1 in 3D, verifiable through the reverse).

All identities are confirmed by constructing unit basis multivectors and computing their products. For example:

```
Multivector::vector(1,0,0)
  .geometric_product(&Multivector::vector(0,1,0))
```

This returns `[0, 0, 0, 0, 1, 0, 0, 0]`, confirming $e_1e_2 = e_{12}$.

IV. Platform Architecture

The platform is structured as a three-tier system optimized for interactive geometric computation. Fig. 2 illustrates the data flow.

A. Frontend: Svelte, CodeMirror, and Three.js

The client is a Svelte 5 application. Users author geometric logic in a CodeMirror editor with switchable R and Python language modes. When the user invokes "Run Code," a regex-based parser extracts vector assignments from the source text and projects them onto the Three.js canvas:

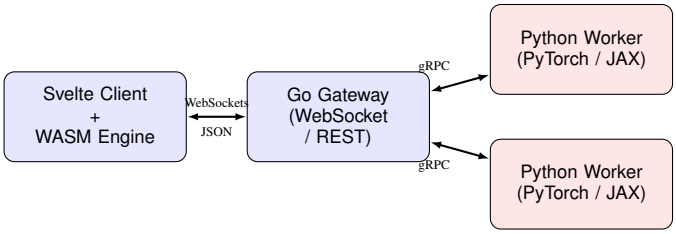

Fig. 2: Tri-partite architecture. The Svelte client performs local multivector arithmetic via the WASM engine and streams geometric state to the Go gateway over WebSockets. The gateway dispatches analysis requests to Python workers over gRPC.

```
// Svelte: Code-to-Canvas regex parser
const terms = line.match(
  /[-+]?\s*[0-9.]*\s*\*?\s*(?:vr\.)?e[123]/g
);
terms.forEach(term => {
  let str = term.replace(/\s/g, '')
                .replace('*', '')
                .replace('vr.', '');
  let val = parseFloat(str);
  if (term.includes('e1')) x += val;
  if (term.includes('e2')) y += val;
  if (term.includes('e3')) z += val;
});
canvasComponent.addExplicitVector(x, y, z);
```

Fig. 3: The Code-to-Canvas parser extracts blade coefficients from R/Python source lines using regex and maps them to 3D vectors on the Three.js canvas. Wedge products (`%^%` in R) trigger bivector rendering.

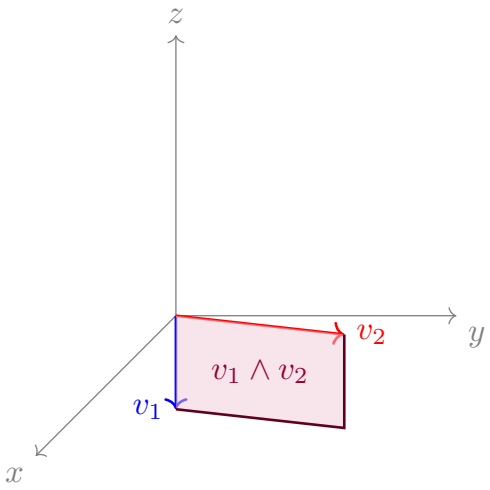

Fig. 4: Native LaTeX visualization of the Three.js canvas state, illustrating two grade-1 vectors v_1, v_2 and their grade-2 wedge product $v_1 \wedge v_2$, which forms the oriented bivector plane.

B. The Monolithic Go Gateway

For inference workloads that exceed the WASM engine's capacity (e.g., running a full E(n)-GNN over a scene graph), the client streams multivector state to a Go server via WebSockets. The Go gateway performs two functions:

1) **REST API with Auth:** Standard HTTP routes wrapped in JWT middleware for user session management [9].
2) **WebSocket-to-gRPC dispatch:** The gateway deserializes incoming JSON payloads into a `Payload` struct and re-encodes them as Protocol Buffer messages for dispatch to Python workers.

The WebSocket payload schema is:

```
// JSON payload from Svelte client
{
  "context": "Canvas",
  "language": "r",
  "objects": [
    { "id": "vecX",
      "data": [0,2,0,0,0,0,0,0] },
    { "id": "vecY",
      "data": [0,0,3,0,0,0,0,0] }
  ]
}
```

Fig. 5: WebSocket message format. Each object's `data` array contains the 8 multivector components in the layout of Eq. 7.

C. The Geometric NC Backend (Python/gRPC)

The analytical backend consists of Python gRPC worker nodes. The service contract is defined in `geometry.proto`:

```
// geometry.proto (abbreviated)
message GeometricState {
  string id = 1;
  repeated float data = 2;
}
message AnalysisRequest {
  string context = 1;
  string language = 2;
  repeated GeometricState objects = 3;
}
service AnalysisService {
  rpc AnalyzeState(AnalysisRequest)
    returns (AnalysisResponse);
  rpc StreamState(stream AnalysisRequest)
    returns (stream AnalysisResponse);
}
```

Fig. 6: The gRPC service definition. `AnalyzeState` handles single-shot inference; `StreamState` supports bidirectional streaming for continuous canvas interaction.

The Python worker currently accepts `AnalyzeState` requests and returns a stub response. The architecture is designed so that swapping in a trained Clifford Neural Layer [4] or E(n)-GNN [3] requires only modifying the `AnalyzeState` handler.

V. Results

A. Engine Correctness

Table I summarizes the verification of the Rust Clifford engine against algebraic identities in $\mathrm{Cl}(3,0)$.

TABLE I: Clifford Engine Verification Results

Identity	Expected	Result
e_1^2	$+1$	✓
$e_1 e_2$	e_{12}	✓
$e_1 e_2 = -e_2 e_1$	Anticommute	✓
$R\, v\, \tilde{R}$ (90° in e_{12})	$3e_2$	✓
$(e_{123})^2$	-1	✓
Wedge: $e_1 \wedge e_2$	e_{12}	✓
Inner: $e_{12} \cdot e_2$	$-e_1$	✓

B. Platform Component Summary

Table II summarizes each architectural layer, its implementation, and current status.

VI. Discussion and Future Work

The current platform demonstrates end-to-end multivector data flow from code editor to 3D canvas to distributed backend. Several extensions are underway.

AST-Based Code Parsing. The current Code-to-Canvas pipeline uses regex heuristics (Fig. 3) that can only extract grade-1 vector assignments. Replacing this with a proper Abstract Syntax Tree (AST) parser would enable the frontend to evaluate full geometric expressions—including rotor sandwiches, wedge products, and user-defined functions—directly in the browser via the WASM engine.

Clifford Neural Layers. The Python gRPC worker is designed to host Clifford Group Equivariant Neural Networks (CGENNs) [4], which parameterize neural network layers as elements of the Clifford group. These layers are $\mathrm{Pin}(p, q, r)$-equivariant by construction, making them a natural fit for the multivector data already flowing through the platform. Implementation will follow the batched tensor architecture of QuAIRKit [11], which demonstrates efficient GPU-accelerated operations over algebraic state spaces.

Conformal Geometric Algebra. The current engine implements $\mathrm{Cl}(3, 0)$, which represents Euclidean 3-space. Extending to the conformal model $\mathrm{Cl}(4, 1)$ would enable circles, spheres, and rigid body motions to be represented as algebraic objects [2], significantly enriching the canvas's geometric vocabulary.

Bivector and Trivector Rendering. The Three.js scene currently renders vectors as arrows. True bivector rendering (oriented discs or parallelograms) and trivector rendering (oriented volumes) would complete the visual correspondence between the algebraic and geometric representations.

Versor Integration. The Versor library (`libvsr`) provides a C++ implementation of GA for arbitrary metric signatures with a focus on conformal geometry and computer graphics [6]. Integrating `libvsr` as a WASM module alongside the current Rust engine would enable the platform to handle projective, conformal, and motor algebras.

VII. Conclusion

We have presented Sensensble Geometry, a web-native lab console that grounds interactive scientific computation in Geometric Algebra. The platform's Rust/WASM Clifford engine provides verified, near-native multivector arithmetic in the browser. Its reactive Svelte frontend and distributed Go/Python backend form a pipeline from pedagogical exploration to production-grade Geometric Neural Computing. By making the mathematics of Clifford Algebra computationally tangible and interactively explorable, the platform contributes to the broader program of "explorable explanations" [10] for scientific communication.

Acknowledgment

The authors thank the developers of `wasm-pack`, `wasm-bindgen`, and the Versor library for their open-source contributions to geometric computing on the web.

References

[1] D. Hestenes and G. Sobczyk, *Clifford Algebra to Geometric Calculus: A Unified Language for Mathematics and Physics*. Dordrecht: Reidel, 1984.

[2] L. Dorst, D. Fontijne, and S. Mann, *Geometric Algebra for Computer Science: An Object-Oriented Approach to Geometry*. San Francisco: Morgan Kaufmann, 2007.

[3] V. G. Satorras, E. Hoogeboom, and M. Welling, "E(n) Equivariant Graph Neural Networks," in *Proc. Int. Conf. Mach. Learn. (ICML)*, 2021, pp. 9323–9332.

[4] D. Ruhe, J. Brandstetter, and P. Forré, "Clifford Group Equivariant Neural Networks," in *Proc. Advances Neural Inf. Process. Syst. (NeurIPS)*, 2023.

[5] J. Diebel, "Representing attitude: Euler angles, unit quaternions, and rotation vectors," Stanford University, Tech. Rep., 2006.

[6] P. Colapinto, "Versor: Spatial computing with conformal geometric algebra," M.S. thesis, Univ. California, Santa Barbara, 2011.

[7] Rust and WebAssembly Working Group, "wasm-pack: Your favorite Rust → Wasm workflow tool," https://rustwasm.github.io/wasm-pack/, 2024.

[8] Rust and WebAssembly Working Group, "wasm-bindgen: Facilitating high-level interactions between Wasm modules and JavaScript," https://rustwasm.github.io/wasm-bindgen/, 2024.

[9] M. Jones, J. Bradley, and N. Sakimura, "JSON Web Token (JWT)," IETF RFC 7519, May 2015.

[10] B. Victor, "Explorable Explanations," 2011. [Online]. Available: http://worrydream.com/ExplorableExplanations/

[11] QuAIRKit Contributors, "QuAIRKit: A Python toolkit for quantum artificial intelligence," https://github.com/QuAIR/QuAIRKit, 2024.

TABLE II: Platform Component Summary

Layer	**Technology**	**Implementation**	**Status**
Clifford Engine	Rust $\rightarrow$ WASM	$\mathrm{Cl}(3,0)$ multivector: geometric, wedge, inner, dual products	Complete, verified (Table I)
Frontend UI	Svelte 5, Three.js, CodeMirror	Code-to-Canvas parser (regex), blade toggles, R/Python language switching	Complete; regex parser handles grade-1 vectors
Go Gateway	Go, gorilla/websocket, gRPC	Monolithic server: REST + Auth + WebSocket-to-gRPC dispatch	Complete, compiles and routes end-to-end
Geometric NC Worker	Python, grpcio, PyTorch	gRPC servicer accepting `AnalyzeState` requests	Scaffold only; neural layers not yet implemented
CI/CD	GitHub Actions	Rust/WASM build, Go build, Python import test	Complete
Notebook Examples	Jupyter (R kernel)	3 notebooks: GA Basics, Electrodynamics, Optoelectronics	Complete